

Claude Code

源码解析报告

基于 2026年3月31日 npm Source Map 泄漏事件的完整技术分析

代码规模: 512,000+ 行 | 文件数量: 1,906 个 | 语言: TypeScript

报告生成时间: 2026年4月1日

目录

1. 事件背景
2. 总体架构
3. 查询引擎与主循环
4. 系统提示词工程
5. 工具系统
6. 多 Agent 蜂群架构
7. 上下文管理与压缩
8. 权限与安全
9. 生态系统
10. 隐藏功能与彩蛋
11. 总结与启示

第一章：事件背景

1.1 泄漏事件时间线

2026年3月31日凌晨，安全研究者 **Chaofan Shou** 发现 Anthropic 发布到 npm 的 Claude Code 包（`@anthropic-ai/claude-code` v2.1.88）中意外打包了 59.8 MB 的 source map 文件，包含完整的 TypeScript 源码。

泄漏事件时间线：

2025-02 首次泄漏 - Claude Code 首次上架 npm，打包了 source map，暴露 13 个月
2026-03-26 CMS 泄漏 - Anthropic CMS 配置错误，暴露 Claude Mythos 模型细节
2026-03-31 npm 泄漏 - v2.1.88 包体积暴涨，512,000 行源码曝光
2026-03-31 紧急响应 - 推送更新、删除旧版本、发起 DMCA

1.2 泄漏技术原理

Source map 文件包含 `sourcesContent` 数组，其中存储了每个原始文件的完整源码。
Bun bundler 默认生成 source map，Anthropic 未在 `.npmignore` 中排除。

```
TypeScript 源码 (1,906 文件)
├── Bun bundler ──> cli.js (打包单文件)
└── 默认生成 ──> cli.js.map (59.8 MB source map)
                    ├── sources: ["../src/main.tsx", ...]
                    └── sourcesContent: ["完整源码...", ...]
                        ↓
                    意外打包进 npm → 任何人可下载
```

1.3 泄漏规模

指标	数值
总代码行数	512,000+
源文件数量	1,906 个 TypeScript/TSX
Source Map 大小	59.8 MB
npm 包体积变化	17 MB → 31 MB
Agent 工具数量	~42 个
Slash 命令数量	~85 个
内部功能开关	44 个

1.4 社区反应与 Anthropic 应对

"The irony is unreal" — Anthropic 宣传 Claude 编写/审查代码能力强大，自家代码因基础错误泄漏。

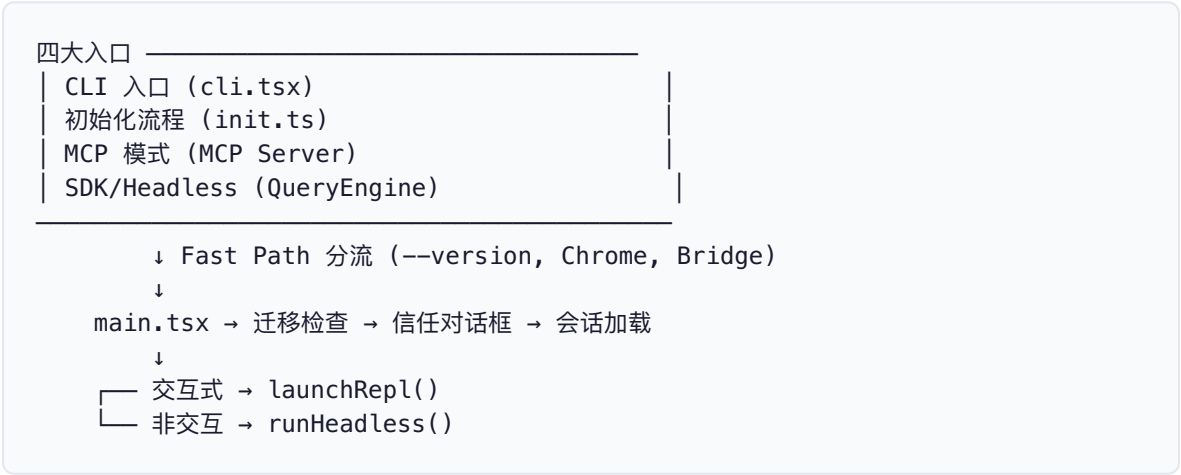
Anthropic 迅速推送更新、删除旧版本、发起 DMCA。官方声明："这是人为造成的发布打包问题。"

第二章： 总体架构

2.1 以 LLM 为内核的操作系统

工具	比喻
Cursor	程序员坐在你旁边，每步点"允许"
Copilot Agent	给程序员虚拟机随便折腾
Claude Code	程序员用你的电脑，配 9 层安检

2.2 四大入口



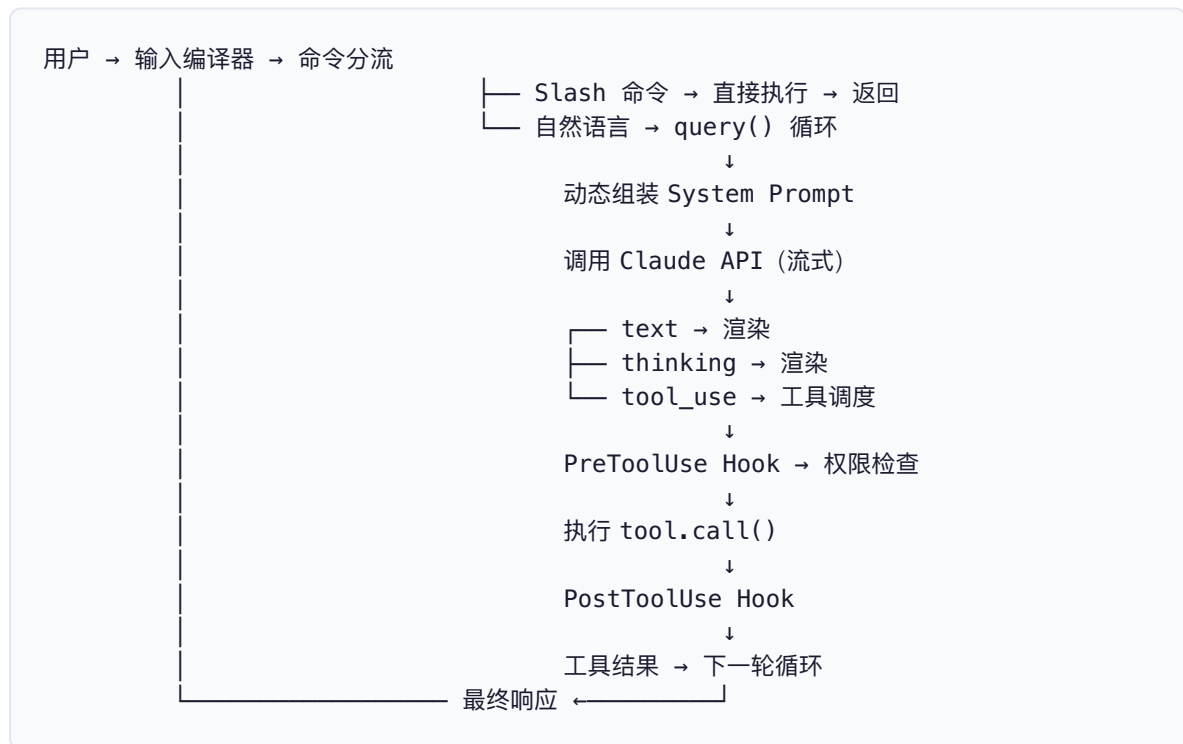
2.3 六层架构

第1层：终端 UI 层 React + Ink, Yoga 布局, 双缓冲, Vim
第2层：主应用层 REPL/Headless, 85+ 命令, 插件/Skills
第3层：查询引擎层 query() 生成器, 流式 API, 压缩, 成本
第4层：工具子系统 42 工具, 权限, 沙箱, Hook
第5层：服务层 Claude API, 遥测, 会话, 记忆, MCP
第6层：传输层 本地 CLI, Bridge, IDE Direct, SSE/WS

2.4 技术栈

类别	技术
语言	TypeScript（严格模式）
运行时	Bun
终端 UI	React + Ink（自定义 reconciler）
布局引擎	Yoga（Flexbox）
API	Claude Messages API + SSE
功能开关	GrowthBook + Statsig
遥测	Datadog + Perfetto

2.5 核心数据流



第三章：查询引擎与主循环

3.1 query() 异步生成器

核心是 `query()` 异步生成器函数，编排每次 AI 交互的完整生命周期：

```
query() 开始
  ↓
Turn 初始化 → Token 用量检查
  ↓
Token ≥ 87%? → 是 → AutoCompact (生成摘要)
  ↓ 否
旧工具结果清理? → 是 → MicroCompact
  ↓
组装 System Prompt → 历史消息过滤 → 工具结果预算
  ↓
调用 Claude API (流式)
  ↓
响应类型分流: text / thinking / tool_use
  ↓ (tool_use)
工具调度 → 执行 (并发/顺序) → 结果注入
  ↓
handleStopHooks()
  ↓
继续? → 有 tool_use → 回到 Token 检查
      → maxTurns / 预算耗尽 / end_turn → 返回 TurnResult
```

3.2 设计哲学

"少搭框架，多信模型"——没有 DAG、没有分类器，模型自己决定下一步做什么。

3.3 Token 预算管理

200K Token 上下文窗口分配：

└─ System Prompt	~15–25K
└─ 对话历史	~100–150K
└─ 工具结果	~20–30K
└─ 响应缓冲	~10–20K

预算控制策略：

└─ +500K 自动续期
└─ 90% 消耗停止
└─ 收益递减检测

第四章：系统提示词工程

4.1 动态组装机制

System Prompt 结构：



4.2 CLAUDE.md 分层加载

~/.claude/CLAUDE.md	← 全局配置（用户偏好）
./CLAUDE.md	← 项目根配置
./.claude/rules/*.md	← 条件规则文件
./子目录/CLAUDE.md	← 子目录特定配置

注：注入到用户消息中（非 System Prompt），保持 cache 稳定性

4.3 行为约束体系

任务哲学：

- 不添加用户没要求的功能
- 不过度抽象，不擅自重构
- 不乱加注释和文档字符串
- 方法失败先诊断再换策略
- 结果如实汇报

工具语法：读文件用 `FileRead` / 改文件用 `FileEdit` / 无依赖并行

Git 安全：不 `push --force` / 不 `reset --hard` / 新 commit 不 `amend`

第五章：工具系统

5.1 42 个工具

类别	工具	说明
文件操作	FileRead, FileWrite, FileEdit	读前写约束
搜索	Grep, Glob, ToolSearch	ripgrep 替代 RAG
执行	Bash, SleepTool	沙箱隔离
代理	Agent, TodoWrite	子代理生成
Web	WebFetch, WebSearch	网页抓取/搜索
MCP	MCPTool	动态工具协议
计划	EnterPlanMode, ExitPlanMode	计划模式
定时	CronCreate/Delete/List	KAIROS

5.2 Fail-closed 设计

`isConcurrencySafe` 和 `isReadOnly` 默认 `false`，忘了声明就视为有风险。

5.3 14 步执行流水线

1. 解析输入 → 2. Schema 校验 → 3. validateInput
→ 4. 投机分类器预判 → 5. PreToolUse Hook
→ 6. 权限决策 (allow/ask/deny/passthrough)
→ 7. 二次修正输入 → 8. tool.call() 执行
→ 9. 遥测日志 → 10. PostToolUse Hook
→ 11. 结构化输出 → 12. readFileState 更新
→ 13. Token 统计 → 14. 结果注入下一轮

5.4 读前写约束

FileRead → 更新 readFileState (文件+mtime)

FileWrite/FileEdit → 检查 readFileState

- |— 已读且 mtime 一致 → 允许
- |— 未读过 → 阻止 (必须先读再写)
- |— mtime 不一致 → 警告

第六章：多 Agent 蜂群架构

6.1 六个内建 Agent

主 Agent (完整权限)

- 通用 Agent (generalPurpose) – 完整工具集
- 探索 Agent (explore) – 只读模式
- 计划 Agent (plan) – 只读模式
- 验证 Agent (verification) – "Try to break it"
- Shell Agent (shell) – 命令执行
- Browser Agent (browser-use) – 浏览器自动化

6.2 Coordinator 协调器

Phase 1: Research → 3 个 Worker 并行搜索
Phase 2: Synthesis → 综合分析
Phase 3: Implementation → 2 个 Worker 串行修改 (避免冲突)
Phase 4: Verification → 验证

6.3 Verification Agent

核心目标：Try to break it

强制运行 Build、测试、Linter。前端浏览器自动化验证，后端 curl 实测。

判决：PASS / FAIL / PARTIAL。与写代码 Agent 利益隔离。

6.4 Fork 缓存优化

统一前缀 `'Fork started - processing in background'`，字节级匹配复用缓存。

第七章：上下文管理与压缩

7.1 三层压缩

第一层 MicroCompact：

旧工具输出 → '[Old tool result content cleared]'

不调 API，本地编辑，提示词主线保留

第二层 AutoCompact：

Token $\geq 87\%$ 触发

预留 13K 缓冲，生成 $\leq 20K$ 摘要

熔断器：连续 3 次失败停止

第三层 Full Compact：

/compact 命令或系统判断

AI 全对话摘要替换历史

CRITICAL：纯文本，禁止调工具

压缩后 $\leq 50K$ Token

7.2 autoDream 夜间记忆蒸馏

触发条件（四项全满足）：

$\geq 24h$ 未整合 + ≥ 5 新会话 + 无并发 + $\geq 10min$ 未扫描

四阶段：

Orient (读取记忆) → Gather (找过时记忆)

→ Consolidate (合并更新) → Prune (≤ 200 行)

第八章：权限与安全

8.1 权限决策模型

```
工具调用 → checkPermissions()  
  |  
  ├── allow - 允许 (可附带 updatedInput)  
  |  
  ├── ask - 询问用户  
  |  
  ├── deny - 直接拒绝  
  |  
  └── passthrough - 始终询问
```

8.2 沙箱机制

```
Bash 执行 → 操作系统判断  
  |  
  ├── Linux → bubblewrap (bwrap) 命名空间隔离  
  |  
  ├── macOS → sandbox-exec 沙箱  
  |  
  └── 可选 → dangerouslyDisableSandbox
```

8.3 Hook 系统

104 个 React Hooks + 运行时事件。四阶段：InstructionsLoaded → UserPromptSubmit → PreToolUse → PermissionRequest。

8.4 Undercover Mode

Anthropic 员工（`USER_TYPE === 'ant'`）贡献开源时自动隐藏 AI 痕迹。

第九章：生态系统

9.1 MCP 协议

MCP 设备连接时操作指南动态注入 System Prompt，工具名 `mcp__server__tool`。

9.2 三套扩展

机制	特点
Skill	轻量 Markdown 工作流
Plugin	重型插件，可接管模型
Workflow	多步骤编排

9.3 Bridge 远程

路径	说明
<code>--remote-control</code>	本地对外暴露
<code>--remote</code>	连接远端
<code>--sdk-url</code>	SDK 传输
<code>useDirectConnect</code>	裸 WebSocket

9.4 TUI 终端

React → 自定义 Reconciler → 虚拟 DOM → Yoga 布局
→ 双缓冲 → Diff → 补丁 → ANSI 输出 (~16ms, ~60fps)

第十章：隐藏功能与彩蛋

10.1 BUDDY 数字宠物

完整宠物收集系统：userId + salt 用 Mulberry32 确定性生成，有稀有度和闪光变体。计划 4 月预告、5 月上线。

10.2 KAIROS 守护进程

后台持续运行，Cron 工具管理定时任务，与 autoDream 联动。

10.3 功能开关（44 个）

开关	说明
PROACTIVE	主动式交互
VOICE_MODE	语音输入
KAIROS	守护进程
BUDDY	数字宠物
COORDINATOR_MODE	协调器

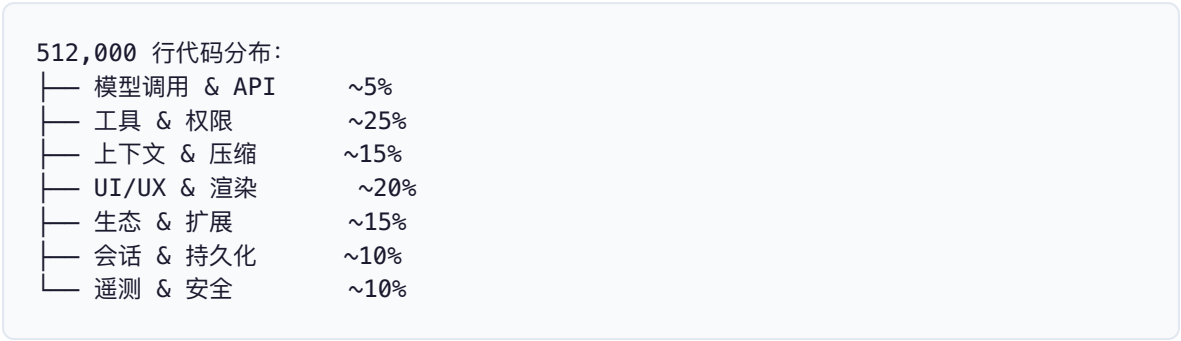
10.4 未发布模型代号

代号	推测
Capybara	下一代推理模型
Fennec	轻量快速模型
Numbat	特殊用途模型

第十一章：总结与启示

11.1 AI Agent 的壁垒

AI Agent 90% 的工作量在 "AI" 之外。



11.2 可借鉴的工程实践

实践	适用场景
Prompt 静态/动态分离	所有 LLM 应用
Fail-closed 安全设计	AI Agent
三层上下文压缩	长对话 AI
Verification Agent	代码生成
读前写约束	文件操作工具
Fork 缓存前缀	多 Agent
autoDream 记忆整合	长期记忆 AI

11.3 最后的思考

没有魔法，只有工程。1,906 个文件、512,000 行代码背后是真实的工程师在解决真实的问题。核心护城河在模型——你可以复制架构，但无法复刻 Claude 的推理能力。

本报告基于公开可获取的信息整理，仅用于技术分析和学习目的。

报告生成时间：2026年4月1日